```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from sklearn import metrics
```

```
mart_dataset = pd.read_csv('/content/Train.csv')
```

```
mart_dataset.head()
```

t	Item_Fat_Content	Item_Visibility	Item_Type	It
0	Low Fat	0.016047	Dairy	2
2	Regular	0.019278	Soft Drinks	
0	Low Fat	0.016760	Meat	1
0	Regular	0.000000	Fruits and Vegetables	1
3	Low Fat	0.000000	Household	

Next steps: Generate code with `mart_dataset` New interactive sheet

```
mart_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8523 entries, 0 to 8522
Data columns (total 12 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Item_Identifier            8523 non-null   object
 1   Item_Weight                7060 non-null   float64
 2   Item_Fat_Content           8523 non-null   object
 3   Item_Visibility            8523 non-null   float64
 4   Item_Type                  8523 non-null   object
 5   Item_MRP                   8523 non-null   float64
 6   Outlet_Identifier          8523 non-null   object
 7   Outlet_Establishment_Year  8523 non-null   int64
 8   Outlet_Size                6113 non-null   object
```

```
 9   Outlet_Location_Type       8523 non-null   object
 10  Outlet_Type                8523 non-null   object
 11  Item_Outlet_Sales          8523 non-null   float64
dtypes: float64(4), int64(1), object(7)
memory usage: 799.2+ KB
```

```
mart_dataset.isnull().sum()
```

	0
Item_Identifier	0
Item_Weight	1463
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	2410
Outlet_Location_Type	0
Outlet_Type	0
Item_Outlet_Sales	0

dtype: int64

```
mart_dataset.shape
```

```
(8523, 12)
```

```
mart_dataset['Item_Weight'].mean()
```

```
np.float64(12.857645184135976)
```

```
mart_dataset['Item_Weight'].fillna(mart_dataset['Item_Weight'].mean(), inplace=
```

```
/tmp/ipykernel_900/1765961367.py:1: FutureWarning: A value is trying to be set c
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  mart_dataset['Item_Weight'].fillna(mart_dataset['Item_Weight'].mean(), inplace
```

```
mode_of_outlet_size = mart_dataset.pivot_table(values='Outlet_Size', columns='C
```

```
print(mode_of_outlet_size)
```

```
Outlet_Type Grocery Store Supermarket Type1 Supermarket Type2 Supermarket Type3
Outlet_Size         Small             Small            Medium            Medium
```

```
from pandas._libs import missing
missing_values = mart_dataset['Outlet_Size'].isnull()
print(missing_values)
```

```
0       False
1       False
2       False
3        True
4       False
        ...
8518    False
8519     True
8520    False
8521    False
8522    False
Name: Outlet_Size, Length: 8523, dtype: bool
```

```
mart_dataset.loc[missing_values, 'Outlet_Size'] = mart_dataset.loc[missing_valu
```

```
mart_dataset.isnull().sum()
```

	0
Item_Identifier	0
Item_Weight	0
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	0
Outlet_Location_Type	0
Outlet_Type	0
Item_Outlet_Sales	0

dtype: int64

```
mart_dataset.describe()
```

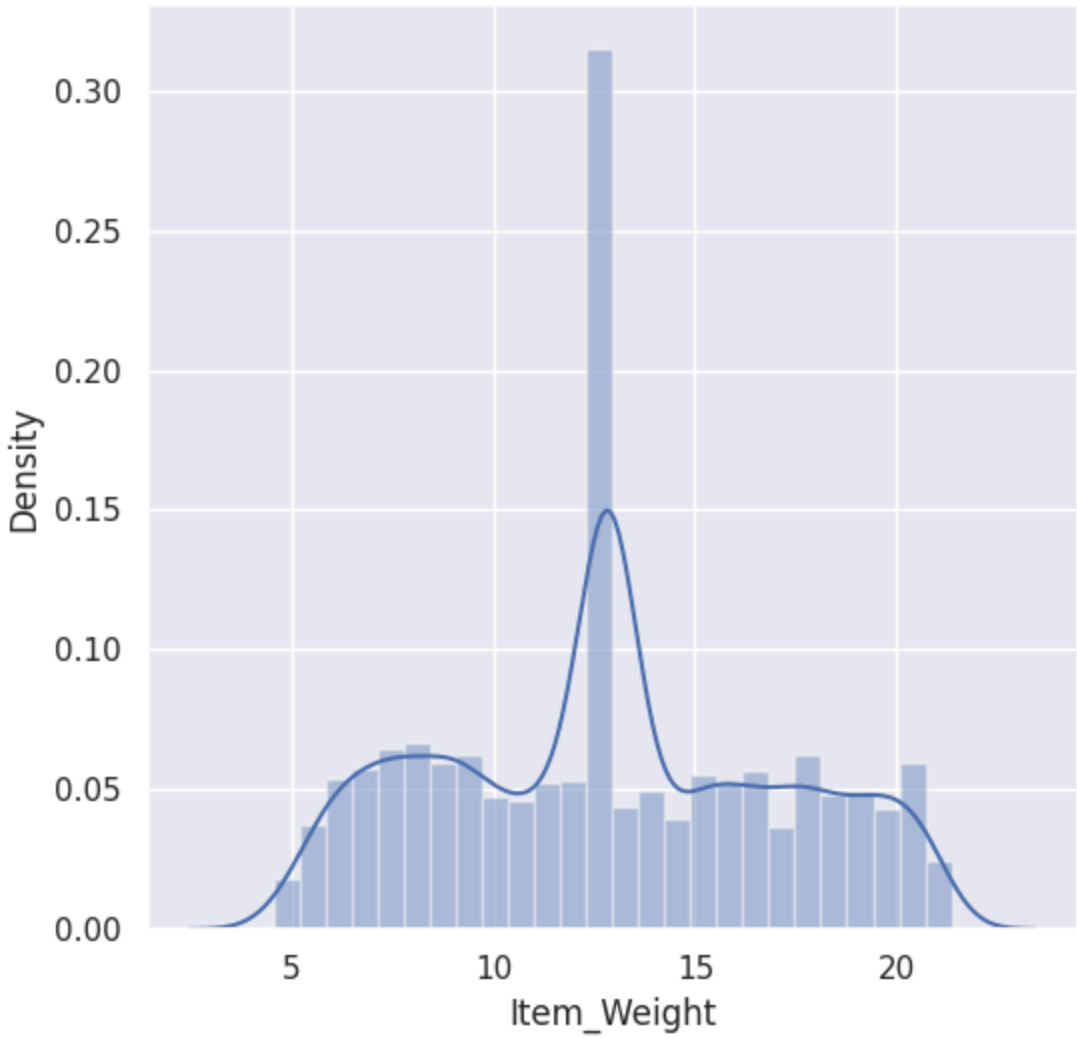
	Item_Weight	Item_Visibility	Item_MRP	Outlet_Establishment_Year	Ite
count	8523.000000	8523.000000	8523.000000	8523.000000	
mean	12.857645	0.066132	140.992782	1997.831867	
std	4.226124	0.051598	62.275067	8.371760	
min	4.555000	0.000000	31.290000	1985.000000	
25%	9.310000	0.026989	93.826500	1987.000000	
50%	12.857645	0.053931	143.012800	1999.000000	
75%	16.000000	0.094585	185.643700	2004.000000	
max	21.350000	0.328391	266.888400	2009.000000	

```
sns.set()
```

```
plt.figure(figsize=(6,6))
sns.distplot(mart_dataset['Item_Weight'])
```

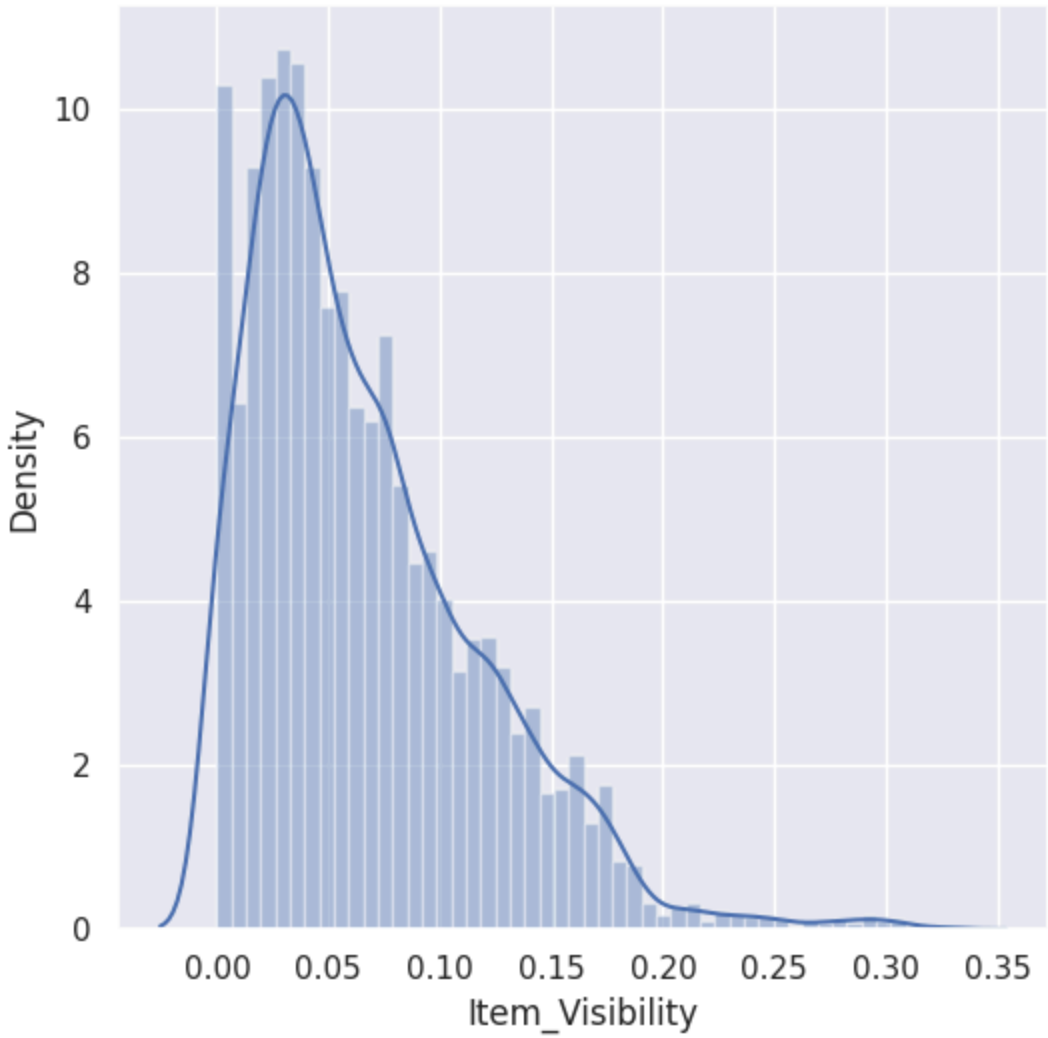
```
plt.show()
```

```
/tmp/ipykernel_900/1350809162.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(mart_dataset['Item_Weight'])
```



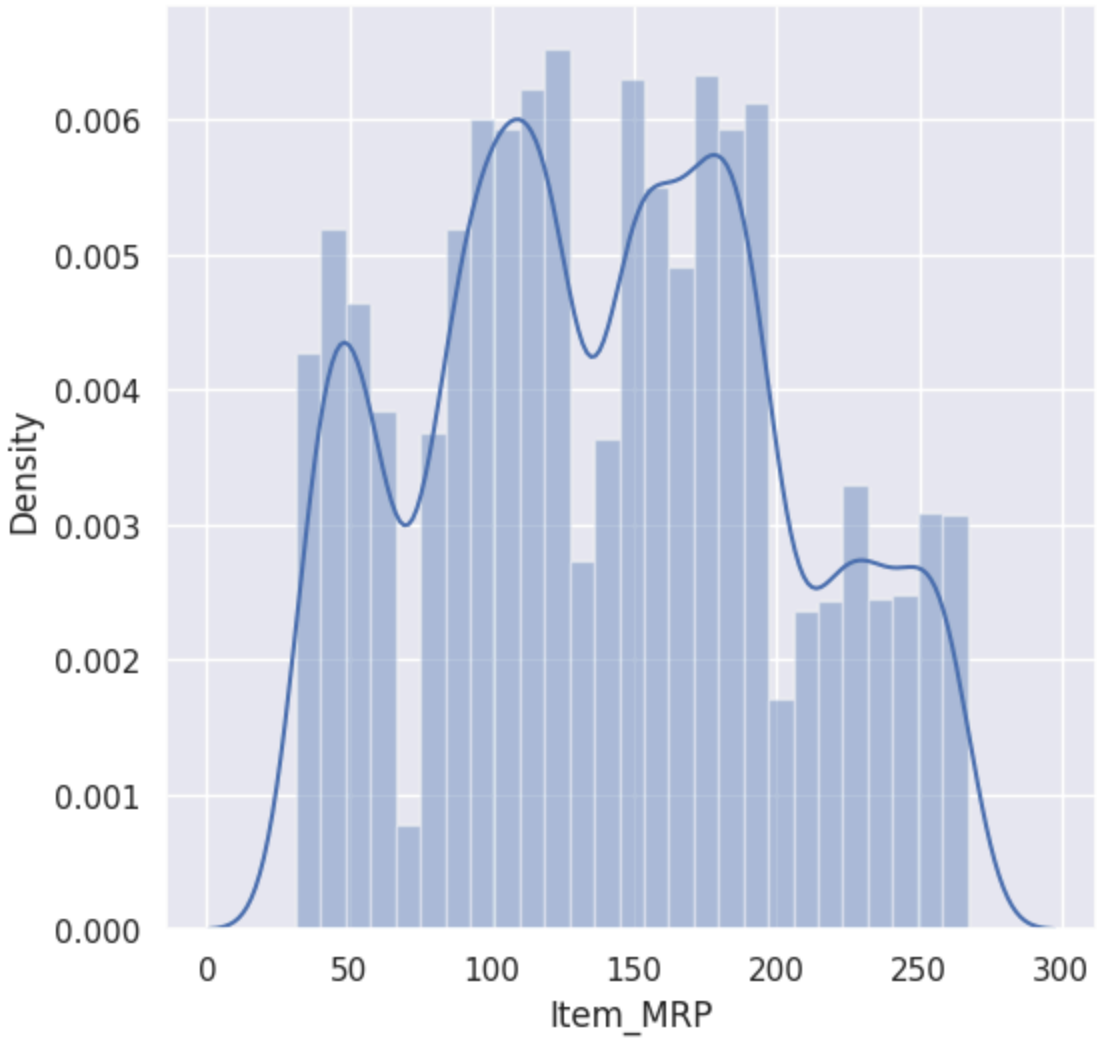

```
plt.figure(figsize=(6,6))
sns.distplot(mart_dataset['Item_Visibility'])
plt.show()
```

```
/tmp/ipykernel_900/3260962778.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(mart_dataset['Item_Visibility'])
```



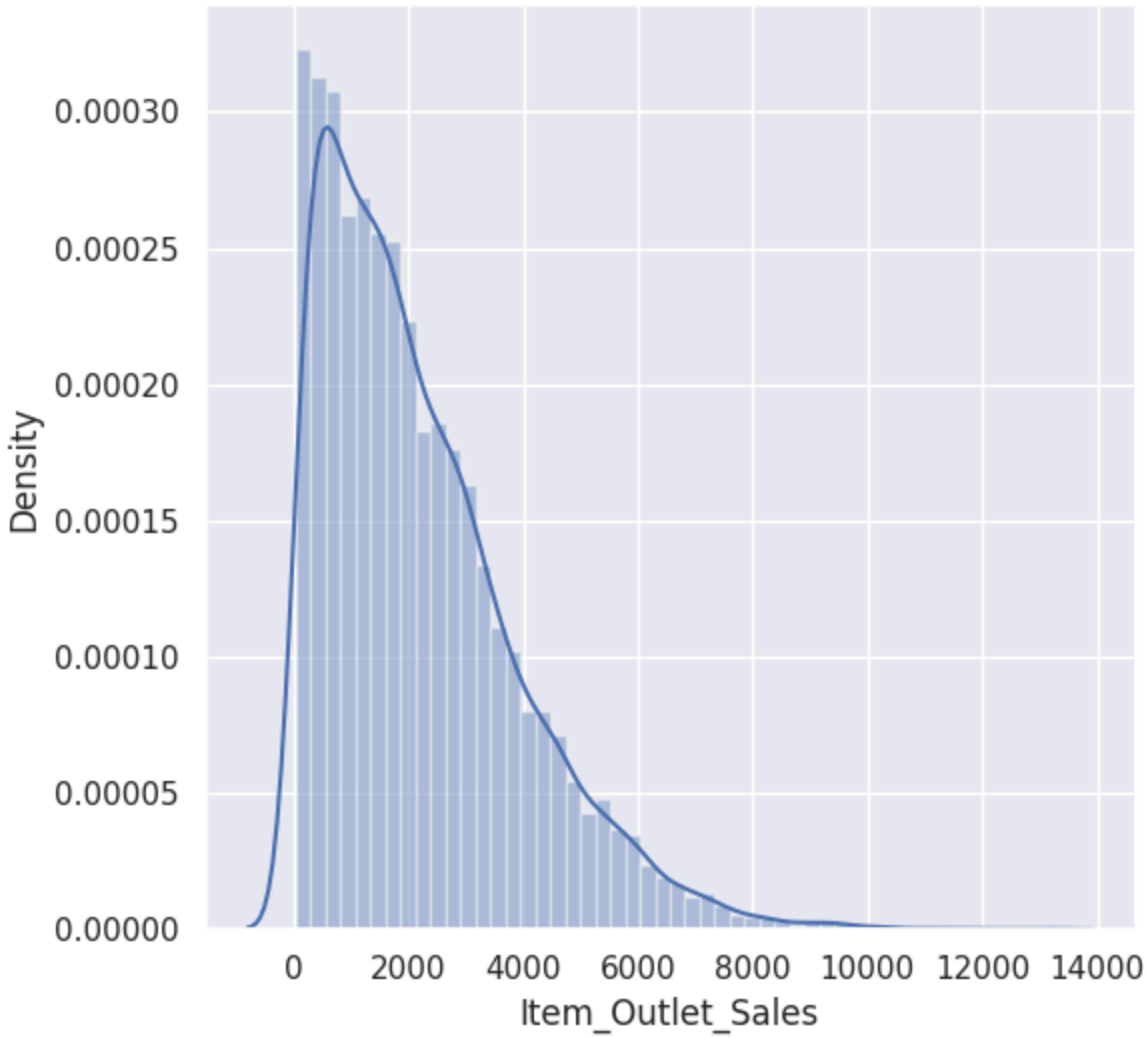

```
plt.figure(figsize=(6,6))
sns.distplot(mart_dataset['Item_MRP'])
plt.show()
```

```
/tmp/ipykernel_900/2493795802.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(mart_dataset['Item_MRP'])
```

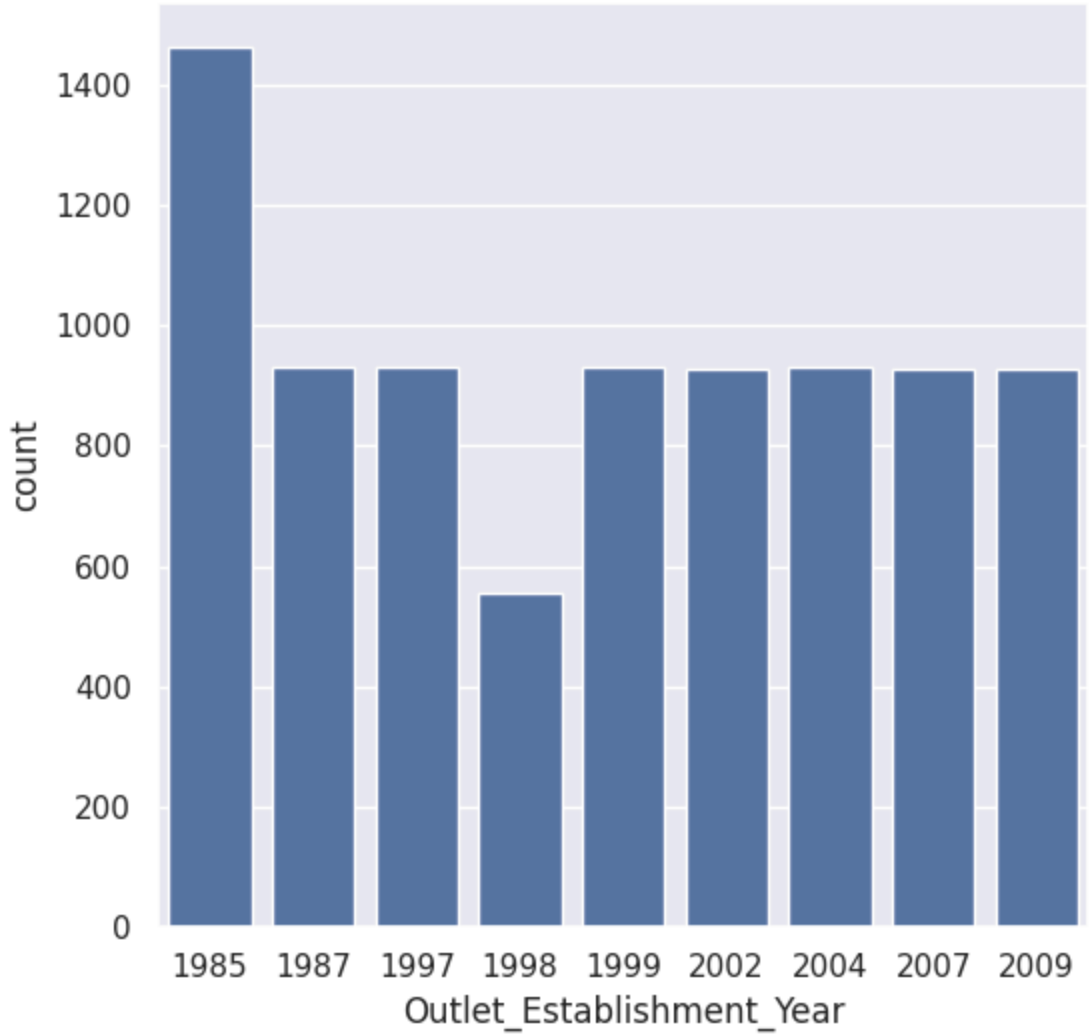



```
plt.figure(figsize=(6,6))
sns.distplot(mart_dataset['Item_Outlet_Sales'])
plt.show()
```

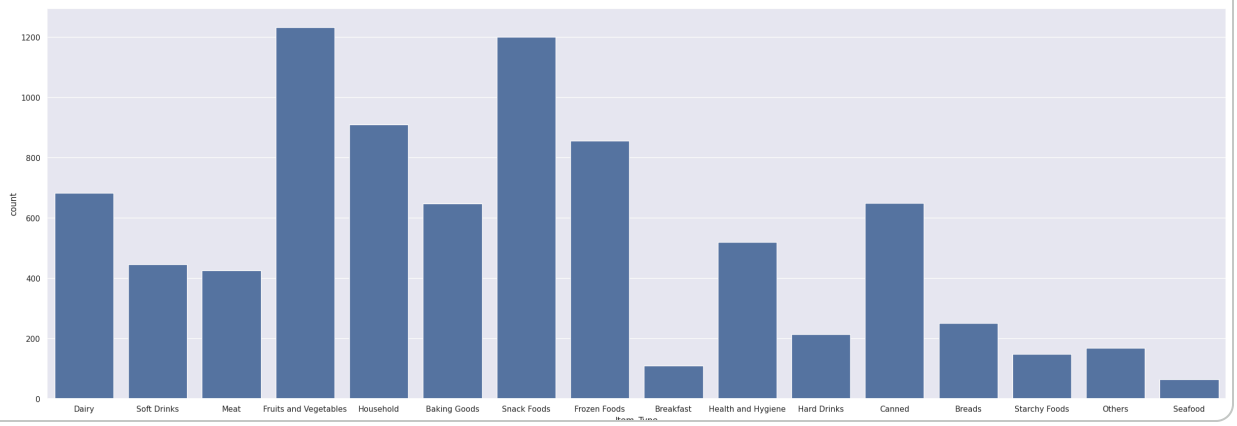
```
/tmp/ipykernel_900/3226448049.py:2: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(mart_dataset['Item_Outlet_Sales'])
```




```
plt.figure(figsize=(6,6))
sns.countplot(x= 'Outlet_Establishment_Year', data=mart_dataset)
plt.show()
```




```
plt.figure(figsize=(30,10))
sns.countplot(x= 'Item_Type', data=mart_dataset)
plt.show()
```




```
mart_dataset['Item_Fat_Content'].value_counts()
```

Item_Fat_Content	count
Low Fat	5089
Regular	2889
LF	316
reg	117
low fat	112

dtype: int64

```
mart_dataset.replace({'Item_Fat_Content': {'LF':'Low Fat', 'reg':'Regular', 'lc
```

```
encoder = LabelEncoder()
```

```
mart_dataset['Item_Identifier'] = encoder.fit_transform(mart_dataset['Item_Ider
```

```
mart_dataset['Item_Identifier'].head()
```

	Item_Identifier
0	156
1	8
2	662
3	1121
4	1297

dtype: int64

```
mart_dataset['Item_Fat_Content'] = encoder.fit_transform(mart_dataset['

mart_dataset['Item_Type'] = encoder.fit_transform(mart_dataset['Item_Ty

mart_dataset['Outlet_Identifier'] = encoder.fit_transform(mart_dataset[

mart_dataset['Outlet_Size'] = encoder.fit_transform(mart_dataset['Outle

mart_dataset['Outlet_Location_Type'] = encoder.fit_transform(mart_datase

mart_dataset['Outlet_Type'] = encoder.fit_transform(mart_dataset['Outle
```

```
mart_dataset.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	I
0	156	9.30	0	0.016047	4	2
1	8	5.92	1	0.019278	14	
2	662	17.50	0	0.016760	10	1
3	1121	19.20	1	0.000000	6	1
4	1297	8.93	0	0.000000	9	

Next steps: Generate code with `mart_dataset` New interactive sheet

```
X = mart_dataset.drop(columns='Item_Outlet_Sales', axis=1)
Y = mart_dataset['Item_Outlet_Sales']
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random
```

```
print(X_train.shape, X_test.shape, X.shape)
```

```
(6818, 11) (1705, 11) (8523, 11)
```

```
regressor_xgb = XGBRegressor()
```

```
regressor_xgb.fit(X_train, Y_train)
```

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=None, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=None,
             n_jobs=None, num_parallel_tree=None, ...)
```

```
training_data_prediction = regressor_xgb.predict(X_train)
```